COMPUTER ENGINEERING

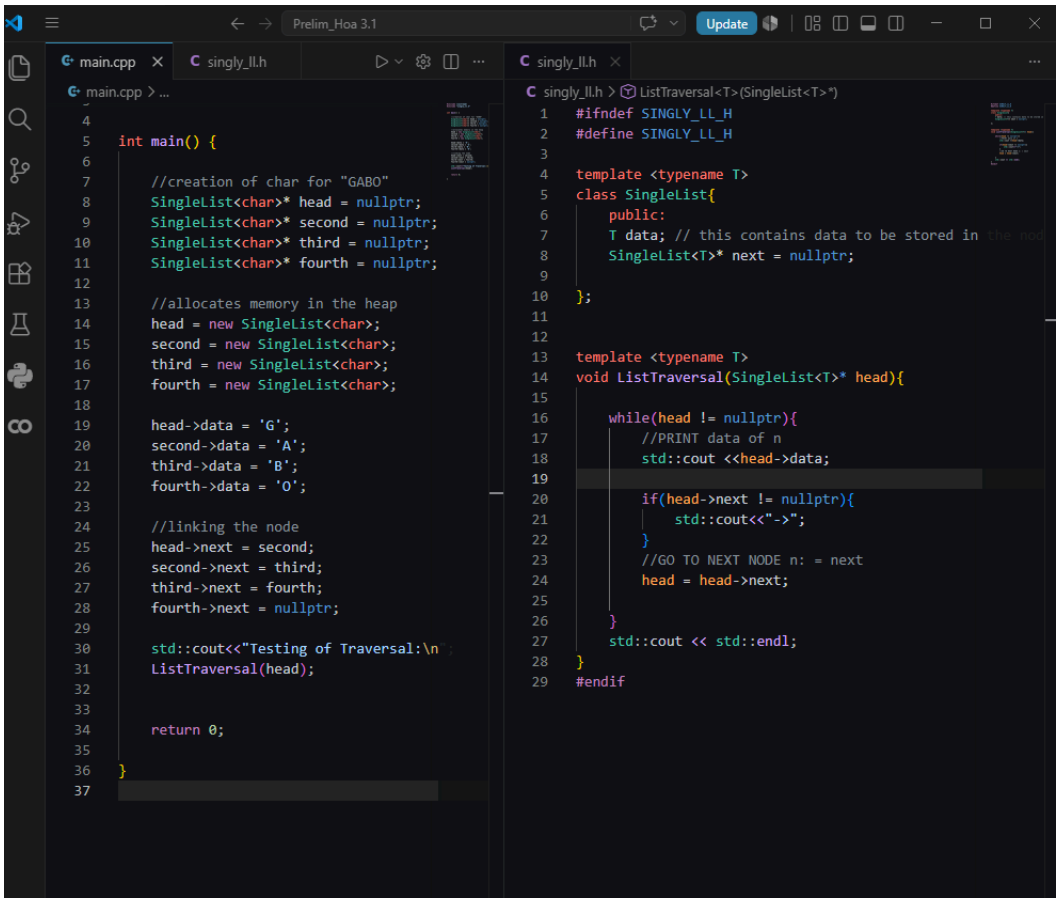
QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures & Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	7/22/26	**DATE SUBMITTED:**	7/22/26
NAME:	Gabriel Dwyane D. Arididdon	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 3.1 **Seatwork 3.1: Creating a Singly Linked List**

1. PROCEDURE OUTPUT

List Traversal



main.cpp

```
int main() {

    //creation of char for "GABO"
    SingleList<char>* head = nullptr;
    SingleList<char>* second = nullptr;
    SingleList<char>* third = nullptr;
    SingleList<char>* fourth = nullptr;

    //allocates memory in the heap
    head = new SingleList<char>;
    second = new SingleList<char>;
    third = new SingleList<char>;
    fourth = new SingleList<char>;

    head->data = 'G';
    second->data = 'A';
    third->data = 'B';
    fourth->data = 'O';

    //linking the node
    head->next = second;
    second->next = third;
    third->next = fourth;
    fourth->next = nullptr;

    std::cout<<"Testing of Traversal:\n
    ListTraversal(head);


    return 0;

}
```

singly_ll.h

```
#ifndef SINGLY_LL_H
#define SINGLY_LL_H

template <typename T>
class SingleList{
    public:
    T data; // this contains data to be stored in the nod
    SingleList<T>* next = nullptr;

};


template <typename T>
void ListTraversal(SingleList<T>* head){

    while(head != nullptr){
        //PRINT data of n
        std::cout <<head->data;

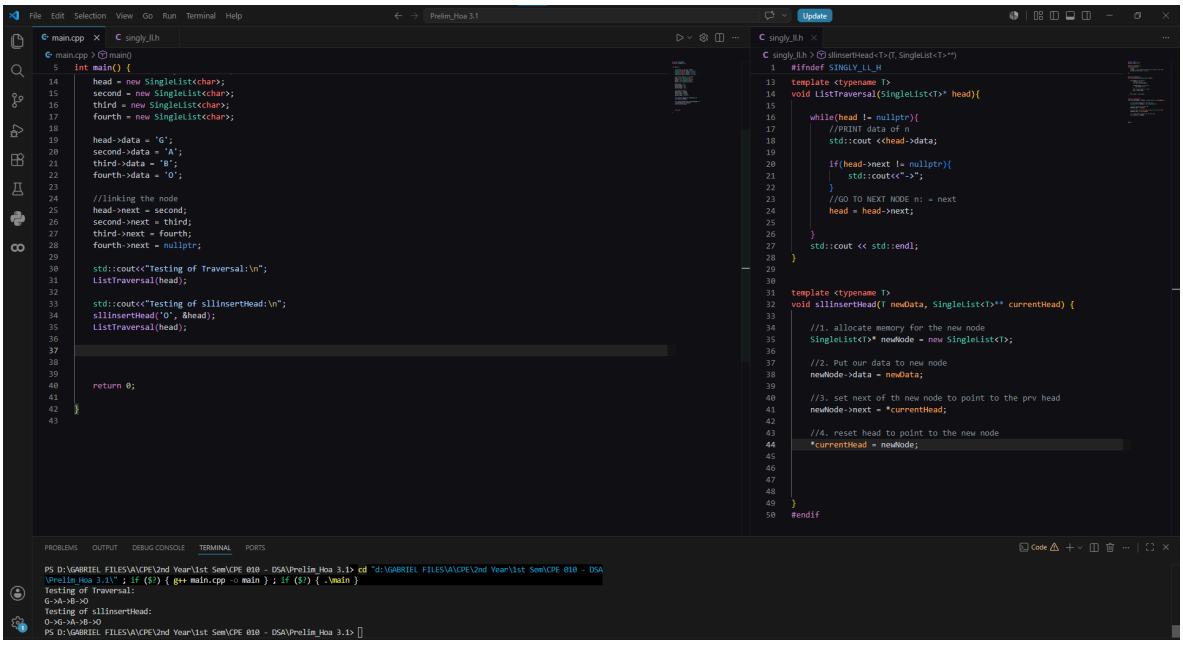
        if(head->next != nullptr){
            std::cout<<"->";
        }
        //GO TO NEXT NODE n: = next
        head = head->next;

    }
    std::cout << std::endl;
}
#endif
```

```
Testing of Traversal:
G->A->B->O
```

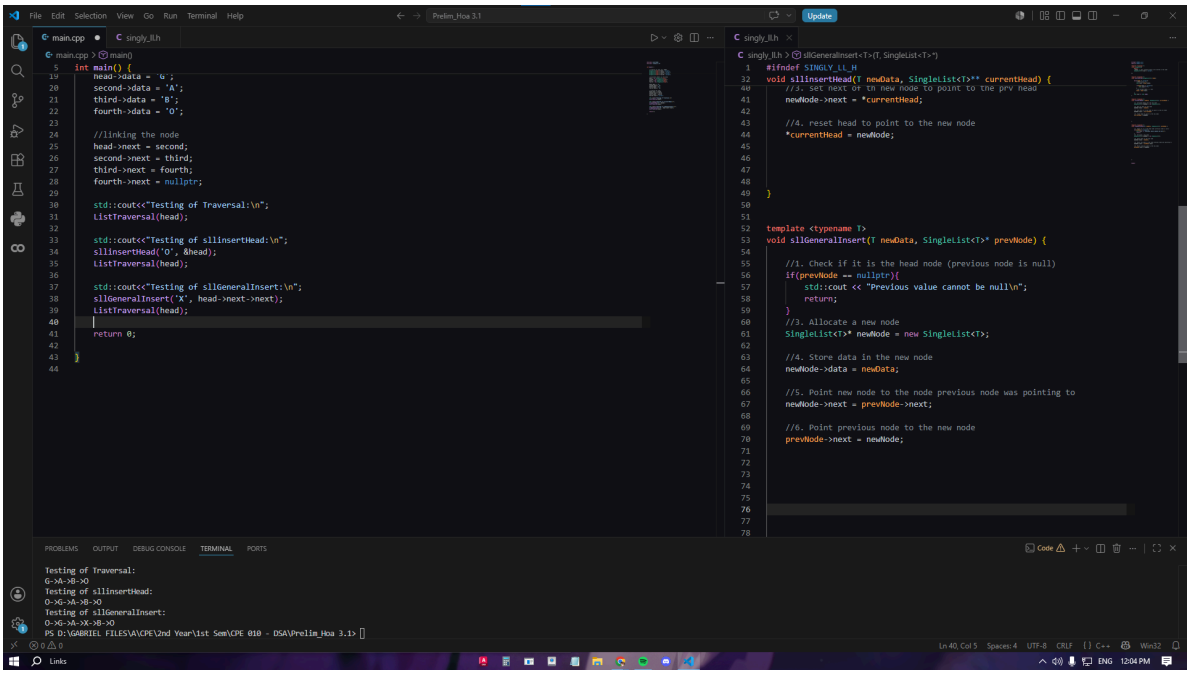
While implementing linked list traversal, I learned how nodes in a singly linked list are accessed differently compared to arrays. Instead of using an index to directly access data, I need to start from the head node and follow the next pointer until I reach the end of the list. I realized that the head pointer is very important because it serves as the entry point of the whole linked list.

Node Insertion at Head



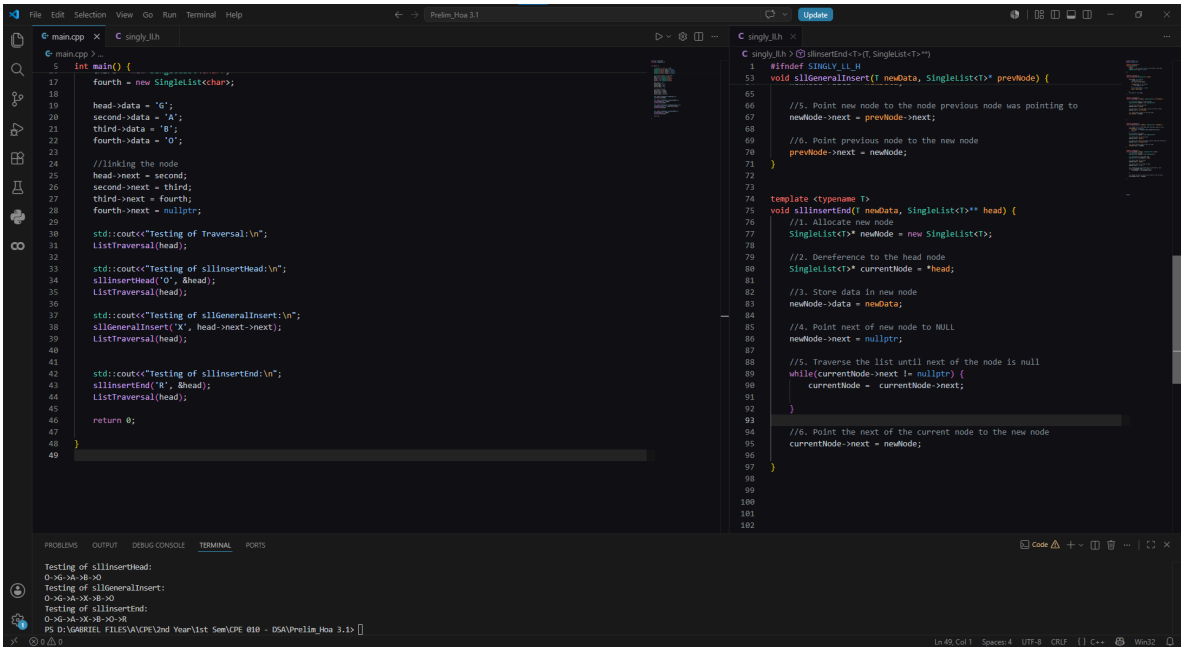
When inserting a node at the head, I learned that this is the simplest and fastest way to add a new node. The process only requires creating a new node, making it point to the current head, and updating the head pointer to the new node. I observed that the original first node does not disappear; instead, it simply becomes the second node after the insertion

General Node Insertion



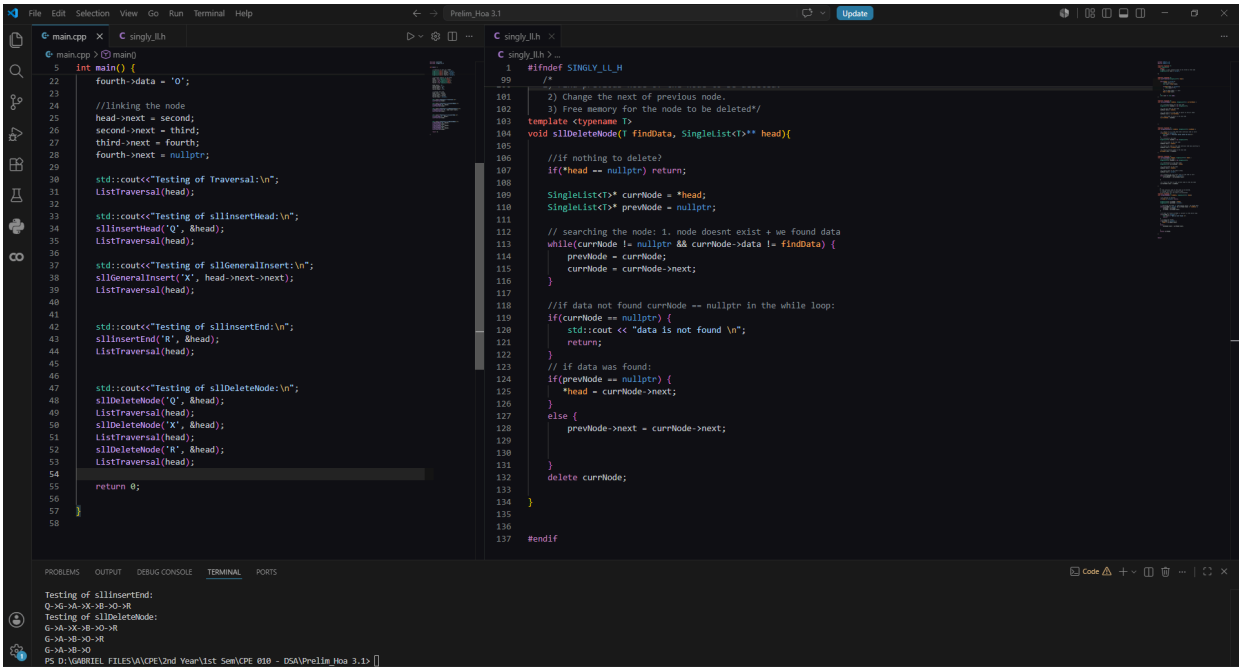
For general node insertion, I learned how a node can be added between existing nodes by using the address of a specific node as the reference point. In the program, using head->next->next means that I am accessing the node sequentially by moving through the next pointers. If I add more ->next, it will continue moving to the next node in the linked list until I reach the desired position for insertion.

Insert Node at the End



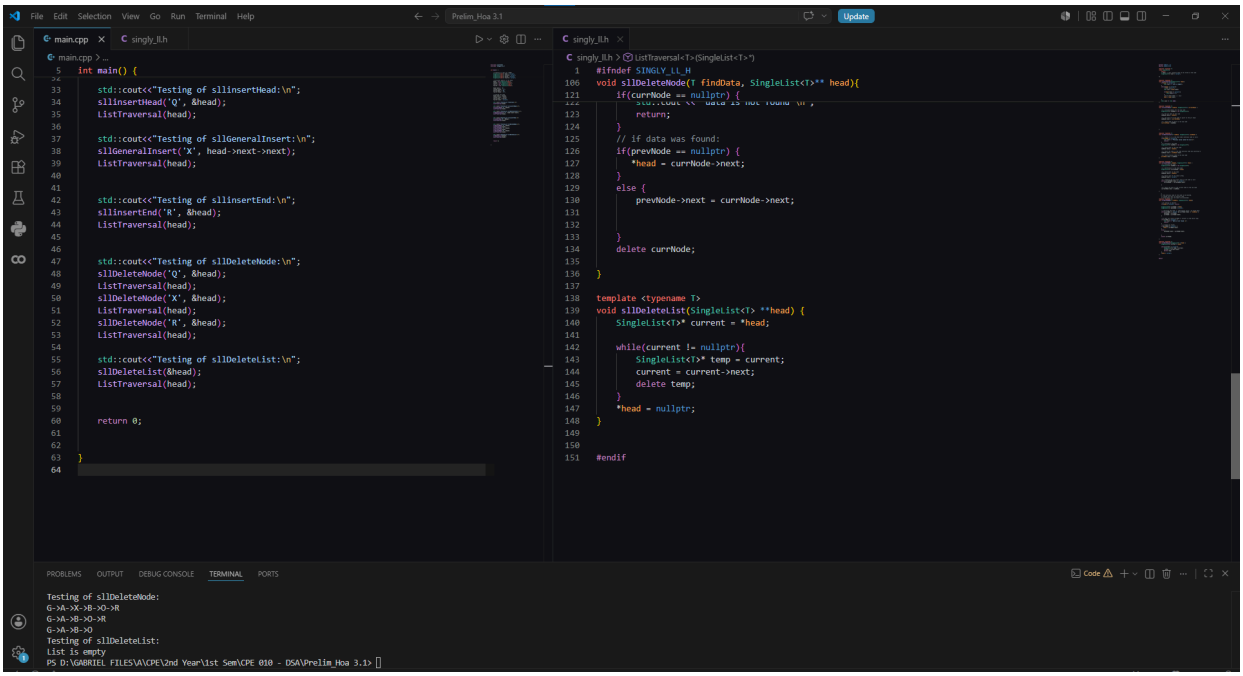
I observed that adding a node at the end requires more steps compared to inserting at the head because it involves searching through the list. However, once the last node is found, the insertion process is simple because the last node only needs to point to the new node.

Delete a Node



I observed that deleting the head node is different because there is no previous node. Instead, the head pointer must be updated to point to the next node.

Delete list



For deleting the entire linked list, I learned how to remove all nodes and properly release the memory that was allocated during the program. Instead of deleting only one node, the function continuously moves through the list and deletes every node until there are no remaining nodes.

Conclusion

To conclude everything, I gained a better understanding of how singly linked lists work and how important pointers are in creating and managing dynamic data structures through this activity/lesson/demo. At first, handling multiple pointers was confusing because every node depends on its connection to the next node. However, by implementing traversal, insertion, and deletion operations, I gradually understood how each pointer change affects the entire list.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.